第三届"兴智杯"全国人工智能创新应用大赛

织序 WeaveOrder

智能信贷风控：可审计的金融智能体

项目名称	织序 WeaveOrder —— 智能信贷风控审批助手
主题赛名称	行业赋能
赛题名称	金融智能体方案赛
团队名称	兰儒恺
填报日期	2026 年 11 月 4 日

中国信息通信研究院等 · 主办 | 中国通信学会金融科技专家委员会等 · 支持

项目基础信息

项目名称	织序 WeaveOrder —— 智能信贷风控审批助手（与官网提交材料保持一致）
应用行业及场景	金融行业 · 信用卡与消费信贷的**授信审批**与**贷后风险监控**。覆盖风险归因、逾期分析、额度与使用率分析、客群分层、账龄趋势五类高频分析场景。
核心技术	☑ 自然语言处理 ☑ 知识图谱 ☑ 深度学习 ☑ 其他：**智能体编排（任务规划 / 工具调用 / 多智能体协同）**、**确定性独立校验**
应用发展阶段	☑ 应用示范阶段（基于 30,000 户真实授信数据完成端到端验证，9 项评测全部通过，产出可运行系统与演示视频）

一、项目内容

1.1 背景及意义

授信审批与贷后监控是信用卡、消费金融等业务的核心环节，具有**高频、刚需、强监管**的特征。传统人工风险分析面临三重困难：

- **数据散**：客户的信用额度、6 期还款状态、6 期账单金额与 6 期还款金额共 25 个字段，风险信号分散在多期行为中，人工难以系统提取。
- **信号弱**：是否违约与「最近一期还款状态」「历史最长逾期」「额度使用率」等维度强相关，但这些衍生指标需要跨期计算，凭经验判断容易漏掉。
- **审批慢**：对一批 30,000 户授信客户做完整风险归因，分析师需经历数据理解、建库、编写调试多段 SQL、交叉核对与报告撰写，标准工时约 **80 分钟**。

与此同时，金融监管对风险模型的**可解释性、可审计性与公平性**提出明确要求：结论必须可追溯到数据、过程可复现、结果可复核。纯人工分析依赖个人经验，难以沉淀与审计；纯大模型生成又存在数字幻觉风险。市场迫切需要一种**既快又可审计**的智能体方案。

本项目以真实信贷数据集为验证对象，构建「织序 WeaveOrder」智能信贷风控智能体，把风险分析从 80 分钟压缩到 **1.5 秒**，并保证报告中的每一个数字都能回溯到数据源。

1.2 主要场景及功能

系统以自然语言为入口，用户直接提出分析需求（如「分析这批授信客户的违约风险，找出高风险客群并归因」），智能体自动完成从需求理解到报告交付的全过程。核心功能如下：

功能	说明	输出
信息理解	大模型把自然语言解析为结构化分析要点（意图、维度、指标），失败自动回退规则解析	brief.json
确定性规划	按意图生成可复现的分析 DAG（步骤、依赖、同层并行策略）	plan.json

功能	说明	输出
系统调用	工具注册表统一调度 file_read / sql_query / report_render，SQL 只读真执行	rows / metrics
独立校验	校验器不走渲染路径，直接从 SQL 结果重新解析每个指标并比对，错即拦截	verify 事件
结果输出	生成 Markdown 风险报告，全部数字引用 SQL 结果	report.md

支持五类分析意图：**违约归因**（主场景）、**逾期分析**、**额度与使用率**、 **客群分层**、**账龄趋势**，覆盖金融机构风险部门的主要日常分析需求。

1.3 实现方案

系统采用「**大模型理解 + 确定性规划 + 工具真执行 + 独立校验**」的五段闭环架构：



数据层：采用 UCI 公开数据集「Default of Credit Card Clients」（台湾地区某银行 2005 年 4－9 月信用卡授信客户，30,000 户、25 个字段），装载进内存 SQLite。所有派生指标（最长逾期月数、近 6 月逾期次数、额度使用率等）**在 SQL 内现算**，不预先固化，保证报告数字可回溯到同一条 SQL。

工具层：工具注册表统一管理三个工具，参数经 JSON Schema 校验；`sql_query` 只允许单条只读 SELECT，并对 ATTACH / PRAGMA / INSERT / UPDATE / DELETE / DROP / CREATE 等关键字做白名单拦截，确保数据源只读、可审计。

规划层：规划器是**确定性模板**——同一意图永远生成同一份 DAG，同层步骤并行执行，因此任意次运行结果完全一致，可复现、可审计。

校验层：校验器不信任渲染层，直接从 SQL 结果重新解析每个指标的引用并独立重算，与报告值比对；一旦不一致立即判定失败并**拒绝发布报告**。

部署与运行：Python 3.9 运行时，零新增第三方依赖（仅用标准库），支持离线规则解析模式与在线大模型模式，可在无网络环境运行，便于私有化部署。

1.4 项目创新点

- **理解与规划分离的分工设计**：语义理解交给大模型，流程规划用确定性模板。既保留自然语言入口的易用性，又避免大模型直接生成流程导致的结果漂移，同输入必得同输出，满足金融监管对可复现的要求。
- **独立校验器实现「数字不可幻觉」**：渲染层算完的每个指标，由校验器从数据源独立重算比对，不一致即拦截、不发布。同一错误注入实验证明：校验开启时错误数字被当场拦截，关闭时错误数字直接发布。
- **风险分层直接可执行**：报告不止给出比率，还输出「高危 / 关注 / 中风险 / 低风险」四档分层及每档的违约占比，可直接映射到审批动作（人工复核、额度冻结、自动通过）。
- **跨场景可复制的引擎原语**：同一套引擎（工具注册表、事件溯源、确定性规划、独立校验）已迁移到差旅编排、经营分析、智慧零售、智能信贷四个场景，迁移时只需更换工具集与数据源，架构与代码主体复

用。

二、应用分析

2.1 市场分析

信用卡与消费金融是典型的**高频、强监管、数据密集**业务。风险分析贯穿授信审批、额度管理、贷后监控与催收策略，是金融机构的核心成本项与价值项。当前市场存在三类方案，均存在明显短板：

方案类型	代表	短板
规则引擎	评分卡、规则库	刚性、维护成本高，难以捕捉多期行为的组合信号
纯大模型	通用大模型问答	数字幻觉、不可审计、结论无法回溯到数据源
BI 分析工具	Tableau / Power BI	需要人工编写查询与解读，效率瓶颈仍在人

本项目的差异化定位是**「可审计的智能体」**：既有自然语言入口的效率，又有确定性规划与独立校验的可信度，正好补上上述三类方案的空白。随着《商业银行互联网贷款管理暂行办法》等监管要求对模型可解释性与审计留痕的强调，「可审计的智能体」具备明确的合规溢价与规模化落地空间。

2.2 商业模式分析

目标客户为银行信用卡中心、消费金融公司、小额贷款公司等持牌金融机构，以及为其提供风控系统的金融科技公司。潜在合作模式包括：

- **私有化部署 + 年度服务费**：将智能体部署在客户内网，与其核心系统、数据仓库、审批工作流集成，按席位与报告量收取年度服务费。
- **与风控系统厂商集成**：作为风险分析模块嵌入现有风控平台，按模块授权收费。
- **监管科技（RegTech）输出**：以「可审计智能体」为卖点，面向中小金融机构提供轻量级 SaaS 分析服务。

市场开发策略：先在单一高价值场景（授信风险归因）建立标杆案例，再横向扩展到贷后监控、反欺诈、合规审查等场景，形成风险分析产品矩阵。

2.3 应用情况

本项目已完成基于**真实公开数据**的端到端验证，并建立了完整的评测与证据体系：

- **真实数据验证**：30,000 户真实授信客户、25 个字段，覆盖额度、还款状态、账单、还款额与违约标签。
- **评测通过率**：9 项评测用例全部通过，其中黄金值由评测脚本 从原始 CSV **独立重算**，与被测实现零共享代码，确保评测本身可信。
- **错误注入对照**：对关键指标注入同一错误，校验开启时拦截、关闭时发布，以对照实验证明校验器的必要性。
- **完整证据链**：运行档案、API 调用日志、数据流转记录、人工基线对比与演示视频全部归档，可逐项核验。

2.4 实际应用效果：人工基线 vs 智能体

以「分析 30,000 户授信客户的违约风险并归因」为对照任务，人工标准作业流程（SOP）实测记录如下：

#	步骤	操作内容	标准工时
1	数据理解	识别 25 个字段与业务语义	8 分钟
2	建库导入	建表并导入 clients 全量数据	5 分钟
3	整体违约率	编写并调试整体违约率 SQL	8 分钟
4	还款状态分组	按最近一月还款状态分组统计违约率	10 分钟
5	历史逾期分组	计算近 6 月最长逾期并分组	12 分钟
6	额度与使用率	额度分档 + 使用率分组统计	12 分钟
7	风险分层	交叉维度分层并核对占比	10 分钟
8	报告撰写	整理数字、撰写发现与结论、排版	15 分钟
	合计	**8 步手工操作**	**80 分钟**

智能体同一任务的实际运行数据：

指标	数值	说明
端到端耗时	**1,477 ms**	含 1 次大模型调用
分析步骤	8 步	全部成功，无人工介入
工具调用	8 次	file_read ×1、sql_query ×6、report_render ×1
报告产出	12 项指标 · 5 张明细表	数字全部来自 SQL 结果
Token 消耗	398 tokens	信息理解 1 次调用（prompt 283 + completion 115）
最终状态	ok	通过独立校验

> **量化成效**：同一分析任务，人工约 **80 分钟**，智能体 **1.5 秒**，效率提升约 **3,200 倍**；更重要的是，人工分析依赖个人经验、难以复现，智能体的确定性规划与独立校验使结果**可复现、可审计、错即拦截**。

2.5 社会效益

- **提升风险防控能力**：把风险分析从「专家经验」沉淀为「可复用的系统能力」，帮助金融机构更早识别高危客群，降低坏账损失。
- **降低重复性人工劳动**：把分析师从数据搬运、SQL 编写与指标核对中解放出来，聚焦于风险策略与决策本身。
- **支撑合规与审计**：确定性规划与独立校验天然具备可解释、可复现、可追溯特征，符合金融监管对模型风险管理与审计留痕的要求。

- **方法论可复制**：同一引擎原语已在差旅、经营分析、零售、信贷四个场景落地，可向更多行业与场景迁移，形成通用的企业流程智能体底座。

三、附录

附录 1 · 运行档案（真实运行截图）

以下为真实运行的完整档案（运行 ID：`180-main`），包含任务输入、信息理解、确定性规划 DAG、工具调用、独立校验与最终报告，所有数字均可回溯。



织序 WeaveOrder · 智能信贷风控运行档案

运行 ID：180-main · 生成时间：2026-11-04T00:26:08

任务输入

自然语言分析需求

分析这批授信客户的违约风险，找出高风险客群并归因

信息理解（大模型）

分析授信客户违约风险，识别高风险客群并进行归因分析

意图：risk_diagnosis　来源：llm

维度：age、education、limit_bal、bill_amt1　指标：default_rate、clients

流程判断（确定性规划）

任务输入 → 信息理解 → 流程判断 → 系统调用 → 结果输出

步骤	目标	工具	依赖
s1	读取信贷数据集（clients 表 30,000 户画像）	file_read	—
s2	整体违约率与客群概览	sql_query	s1
s3	最近一月还款状态分组违约率（最强信号）	sql_query	s1
s4	历史最长逾期月数分组违约率	sql_query	s1
s5	额度分档违约率	sql_query	s1
s6	额度使用率分组违约率	sql_query	s1
s7	风险分层（高危/关注/中风险/低风险）	sql_query	s1
s8	生成信贷风险归因报告（指标全部引用 SQL 结果）	report_render	s2, s3, s4, s5, s6, s7

本档案由 runs/ 下的 trace/plan/result 自动生成，数字均可回溯。

运行档案（第 1 段）：任务输入 → 信息理解 → 流程判断 DAG

系统调用（工具注册表 · 真执行）

#	步骤	工具	参数	时刻
5	s1	`file_read`	–	1207 ms
13	s2	`sql_query`	`SELECT COUNT(*) AS clients, SUM(is_default) AS defaults, ROUND(SUM(is_…`	1440 ms
14	s3	`sql_query`	`SELECT CASE WHEN pay_0<=-1 THEN '正常/循环(-2,-1)' WHEN pay_0=0 THEN '最低还款…`	1440 ms
15	s4	`sql_query`	`SELECT CASE WHEN mx<0 THEN '正常/循环' WHEN mx=0 THEN '最低还款' WHEN mx=1 THE…`	1440 ms
16	s5	`sql_query`	`SELECT CASE WHEN limit_bal<=50000 THEN '≤5万' WHEN limit_bal<=100000 TH…`	1440 ms
20	s6	`sql_query`	`SELECT CASE WHEN util<0 THEN '负值(溢缴)' WHEN util<0.2 THEN '0–20%' WHEN …`	1442 ms
24	s7	`sql_query`	`SELECT CASE WHEN pay_0>=2 THEN '高危(最近逾期≥2期)' WHEN pay_0=1 THEN '关注(最近逾…`	1447 ms
34	s8	`report_render`	–	1477 ms

通过 **确定性校验器**

全部指标与数据源一致

运行指标

1477 ms	8	8	398	ok
端到端耗时	分析步骤	工具调用	Tokens 消耗	最终状态

本档案由 runs/ 下的 trace/plan/result 自动生成，数字均可回溯。

运行档案（第 2 段）：系统调用 → 独立校验 → 运行指标

结果输出

授信客户风险分析 · 违约归因

- **整体违约率**：22.1%
- **授信客户数**：30,000
- **违约客户数**：6,636
- **平均授信额度**：167,484
- **平均额度使用率**：42.4%
- **正常/循环违约率**：15.6%
- **最近逾期2期违约率**：69.1%
- **最近逾期≥3期违约率**：71.9%
- **最低还款违约率**：10.8%
- **历史逾期≥3期违约率**：62.9%
- **额度≤5万违约率**：31.8%
- **额度>40万违约率**：12.0%
- **使用率≥80%违约率**：26.7%

最近一月还款状态分组违约率

pay0_band	clients	default_rate
正常/循环(-2,-1)	8445	15.62
最低还款(0)	14737	12.81
逾期1期	3688	33.95
逾期2期	2667	69.14
逾期3期及以上	463	71.92

历史最长逾期月数分组违约率

overdue_band	clients	default_rate
正常/循环	5195	14.21
最低还款	14736	10.83
逾期1期	1689	24.99
逾期2期	7187	43.55
逾期3期及以上	1193	62.87

额度分档违约率

limit_band	clients	default_rate	limit_avg
≤5万	7676	31.79	35205.0
5–10万	4822	25.8	80759.0
10–20万	7880	19.48	158327.0
20–40万	7818	15.35	286788.0
>40万	1804	12.03	485116.0

额度使用率分组违约率

util_band	clients	default_rate
负值(溢缴)	590	18.47
0–20%	12665	18.29
20–50%	4371	21.07
50–80%	4394	26.33
≥80%	7980	26.73

风险分层

risk_tier	clients	default_rate	default_share
高危(最近逾期≥2期)	3130	69.55	32.81
关注(最近逾期1期)	3688	33.95	18.87
中风险(历史逾期≥2期)	3251	26.85	13.16
低风险(无/轻微逾期)	19931	11.71	35.17

发现

- 整体违约率 22.12%（6,636/30,000），客群风险分化明显，需要分层管理而非一刀切审批
- 「最近一月还款状态」是最强的风险信号：正常/循环客群违约率 15.62%，而最近逾期 2 期者 69.14%、逾期 ≥3 期者 71.92%，相差约 4.5 倍
- 历史最长逾期月数与违约率单调正相关：逾期 ≥3 期 62.87% vs 最低还款 10.83%，相差 5.8 倍
- 低额度客群违约率显著更高（≤5 万档 31.79% 约为 >40 万档 12.03% 的 2.6 倍），说明额度本身承载了风险筛选信息
- 额度使用率 ≥80% 客群违约率 26.73%，高于 0–20% 档的 18.29%，高使用率是资金链紧张的直接体现

结论

风险由「近期还款状态 + 历史逾期深度 + 额度水平」共同决定：建议对最近逾期 ≥2 期、历史逾期 ≥3 期、使用率 ≥80% 的客户触发人工复核与额度冻结，对正常还款且低使用率的客户可自动化审批并考虑提额。

本档案由 runs/ 下的 trace/plan/result 自动生成，数字均可回溯。

运行档案（第 3 段）：结果输出（12 项指标、5 张明细表，数字全部来自 SQL 结果）

附录 2 · 演示视频（扫码观看）

演示视频完整展示系统架构、真实运行与结果归因，时长约 93 秒，扫码即可观看：

https://raw.githubusercontent.com/iakRulan/weaveorder-agent/main/docs/180-demo.mp4

附录 3 · 评测报告（黄金值独立重算）

9 项评测用例全部通过。黄金值由评测脚本从原始 CSV 独立重算，与被测实现零共享代码。

Task	场景	期望	实际	结果
F01	违约归因（主用例）	ok	ok	PASS
F02	逾期维度	ok	ok	PASS
F03	额度与使用率	ok	ok	PASS
F04	客群分层	ok	ok	PASS
F05	账龄趋势	ok	ok	PASS
F06	综合概览	ok	ok	PASS
F07	注入错误 + 校验开	failed	failed	PASS
F08	注入错误 + 校验关	ok	ok	PASS
F09	跨维度一致性	ok	ok	PASS

关键黄金值（独立重算）：

指标	数值	指标	数值
授信客户数	30,000 户	违约客户数	6,636 户
整体违约率	22.12%	平均授信额度	167,484
最近正常/循环违约率	15.62%	最近逾期 2 期违约率	69.14%
最近逾期 ≥3 期违约率	71.92%	最低还款违约率	10.83%
历史逾期 ≥3 期违约率	62.87%	额度 ≤5 万违约率	31.79%
额度 >40 万违约率	12.03%	使用率 ≥80% 违约率	26.73%

附录 4 · 风险归因结果（主场景输出）

系统对 30,000 户授信客户输出的核心归因（报告原文摘要）：

客群（按最近一月还款状态）	客户数	违约率	相对整体
正常 / 循环	8,445	15.62%	−29%
最低还款	14,737	12.81%	−42%
逾期 1 期	3,688	33.95%	+53%
逾期 2 期	2,667	69.14%	+213%
逾期 ≥3 期	463	71.92%	+225%

风险分层结果可直接映射审批动作：

风险档	判定条件	客户数	违约率	占全部违约	建议动作
高危	最近逾期 ≥2 期	3,130	69.55%	32.81%	人工复核 + 额度冻结
关注	最近逾期 1 期	3,688	33.95%	18.87%	人工复核
中风险	历史逾期 ≥2 期	3,251	26.85%	13.16%	加强监控
低风险	无/轻微逾期	19,931	11.71%	35.17%	自动审批

> 结论：风险由「近期还款状态 + 历史逾期深度 + 额度水平」共同决定。建议对最近逾期 ≥2 期、历史逾期 ≥3 期、使用率 ≥80% 的客户触发人工复核与额度冻结；对正常还款且低使用率的客户可自动化审批并考虑提额。

附录 5 · 证据材料清单

材料	文件	说明
运行档案（网页）	docs/180-evidence/run-archive.html	含 trace / plan / report 全量事件
运行档案截图	docs/180-evidence/run-archive-part1/2/3.png	附录 1 所用截图
API 调用日志	docs/180-evidence/api-call-logs.md	8 次工具调用与完整事件序列
数据流转记录	docs/180-evidence/data-flow.md	数据源、派生指标口径、步骤依赖
人工基线对比	docs/180-evidence/comparison.md	80 分钟 vs 1.5 秒量化对照
演示视频	docs/180-demo.mp4	93 秒，二维码见附录 2
评测报告	eval/results-finance.md / .json	9/9 通过，黄金值独立重算
系统源码	finance_agent.py / core/finance.py	可运行、可复现

附录 6 · 核心代码摘录

独立校验器（不信任渲染层，从 SQL 结果独立重算每个指标）：

```
def _verify_report(self, out):
    for metric in out.get("metrics") or []:
        expected = metric.get("current")
        if metric.get("baseline") is not None:
            base = metric.get("baseline")
            expected = (expected - base) / base * 100
        if abs(float(metric.get("value", 0)) - float(expected)) > 0.01:
            return False, f"「{metric['name']}」报告值与数据源重算值不一致，已拦截"
    return True, "全部指标与数据源一致"
```

只读 SQL 白名单（数据源不可被写入）：

```
for kw in ("ATTACH", "DETACH", "PRAGMA", "INSERT", "UPDATE", "DELETE", "DROP", "CREATE"):
    if kw in cleaned.upper():
        raise ValueError(f"检测到受限关键字：{kw}")
```

确定性规划（同一意图生成同一份 DAG，同层并行执行）：

```
plan = build_plan(brief)          # 按意图生成 DAG
ready = [s for s in pending if all(statuses.get(d, {}).get("status") in ("ok","degraded")
                                   for d in s["depends_on"])]
with ThreadPoolExecutor(max_workers=min(4, len(ready))) as pool:
    outcomes = dict(zip([s["id"] for s in ready], pool.map(run_step, ready)))
```

附录 7 · 结语与获取方式

织序 WeaveOrder 已在**差旅编排、经营分析、智慧零售、智能信贷**四个场景完成落地验证，形成可复用的企业流程智能体底座：同一套引擎原语（工具注册表、事件溯源、确定性规划、独立校验），只需更换工具集与数据源即可迁移到新的行业场景，迁移成本低、可复制性强。

完整源码、评测报告与运行档案已开源，欢迎扫码观看演示视频或访问仓库获取：

- 代码仓库：https://github.com/iakRulan/weaveorder-agent
- 演示视频：https://raw.githubusercontent.com/iakRulan/weaveorder-agent/main/docs/180-demo.mp4
- 评测报告：eval/results-finance.md（9/9 通过，黄金值由脚本独立重算）
- 联系人：兰儒恺（武汉科技大学） · 18570368575